Chapter 5. Replicated Load-Balanced Services

The simplest distributed pattern, and one that most are familiar with, is a replicated load-balanced service. In such a service, every server is identical to every other server and all are capable of supporting traffic. The pattern consists of a scalable number of servers with a load balancer in front of them. The load balancer is typically either completely round-robin or uses some form of session stickiness. The chapter will give a concrete example of how to deploy such a service in Kubernetes.

Stateless Services

Stateless services are ones that don't require saved state to operate correctly. In the simplest stateless applications, even individual requests may be routed to separate instances of the service (see Figure 5-1). Examples of stateless services include things like static content servers and complex middleware systems that receive and aggregate responses from numerous different backend systems.

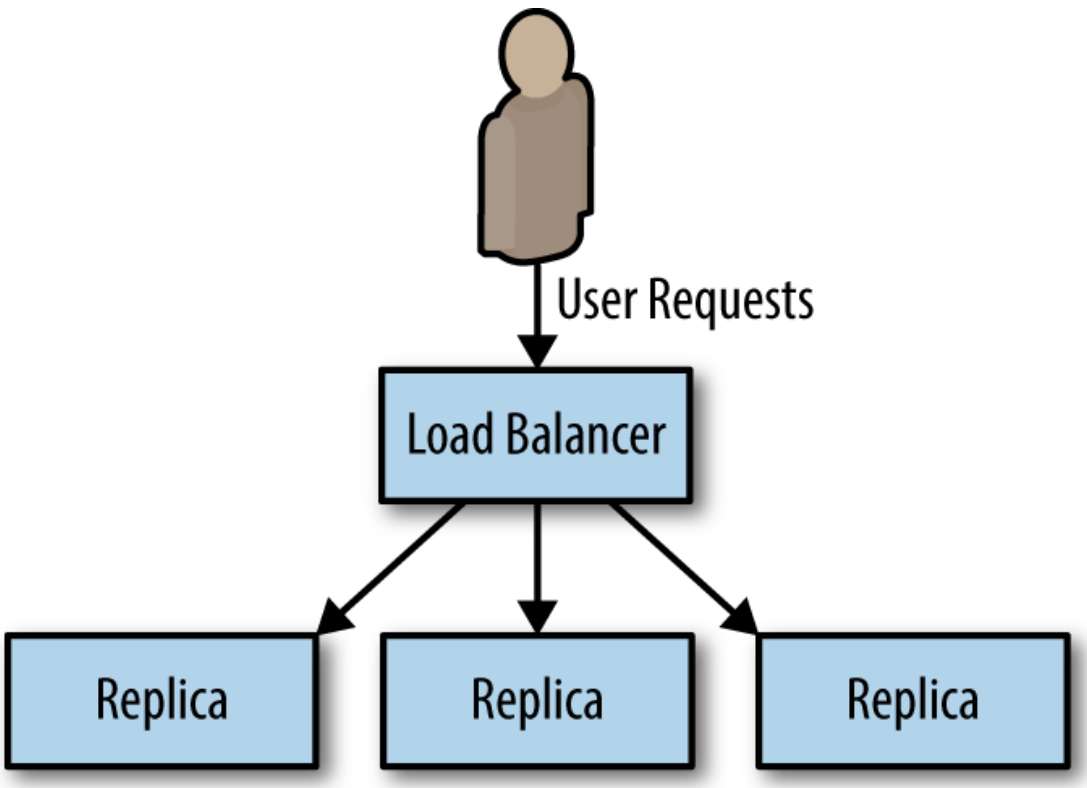

Figure 5-1. Basic replicated stateless service

Stateless systems are replicated to provide redundancy and scale. No matter how small your service is, you need at least two replicas to provide a service with a "highly available" service level agreement (SLA). To understand why this is true, consider trying to deliver a three-nines (99.9% availability). In a *three-nines service*, you get 1.4 minutes of downtime per day (24 × 60 × 0.001). Assuming that you have a service that never crashes, that still means you need to be able to do a software upgrade in less than 1.4 minutes in order to hit your SLA with a single instance. And that's assuming that you do daily software rollouts. If your team is really embracing continuous delivery and you're pushing a new version of software every hour, you need to be able to do a software rollout in 3.6 *seconds* to achieve your 99.9% uptime SLA with a single instance. Any longer than that and you will have more than 0.01% downtime from those 3.6 seconds.

Of course, instead of all of that work, you could just have two replicas of your service with a load balancer in front of them. That way, while you are doing a rollout, or in the—unlikely, I'm sure—event that your software crashes, your users will be served by the other replica of the service and never know anything was going on.

As services grow larger, they are also replicated to support additional users. *Horizontally scalable* systems handle more and more users by adding more replicas; see Figure 5-2. They achieve this with the load-balanced replicated serving pattern.

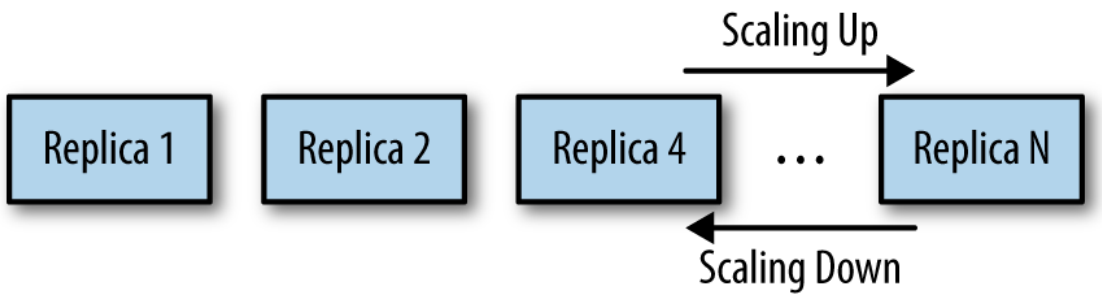

Figure 5-2. Horizontal scaling of a replicated stateless application

Readiness Probes for Load Balancing

Of course, simply replicating your service and adding a load balancer is only part of a complete pattern for stateless replicated serving. When designing a replicated service, it is equally important to build and deploy a readiness probe to inform the load balancer. We have discussed how

health probes can be used by a container orchestration system to determine when an application needs to be restarted. In contrast, a *readiness probe* determines when an application is ready to serve user requests. The reason for the differentiation is that many applications require some time to become initialized before they are ready to serve. They may need to connect to databases, load plugins, or download serving files from the network. In all of these cases, the containers are *alive*, but they are not *ready*. When building an application for a replicated service pattern, be sure to include a special URL that implements this readiness check.

Hands On: Creating a Replicated Service in Kubernetes

The instructions below give a concrete example of how to deploy a stateless, replicated service behind a load balancer. These directions use the Kubernetes container orchestrator, but the pattern can be implemented on top of a number of different container orchestrators.

To begin with, we will create a small NodeJS application that serves definitions of words from the dictionary.

To try this service out, you can run it using a container image:

```
docker run -p 8080:8080 brendanburns/dictionary-server
```

This runs a simple dictionary server on your local machine. For example, you can visit *http://localhost:8080/dog* to see the definition for *dog*.

If you look at the logs for the container, you'll see that it starts serving immediately but only reports readiness after the dictionary (which is approximately 8 MB) has been downloaded over the network.

To deploy this in Kubernetes, you create a `Deployment`:

```
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: dictionary-server
spec:
  replicas: 3
```

```
  template:
    metadata:
      labels:
        app: dictionary-server
    spec:
      containers:
      - name: server
        image: brendanburns/dictionary-server
        ports:
        - containerPort: 8080
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
          initialDelaySeconds: 5
          periodSeconds: 5
```

You can create this replicated, stateless service with:

```
kubectl create -f dictionary-deploy.yaml
```

Now that you have a number of replicas, you need a load balancer to bring requests to your replicas. The load balancer serves to distribute the load as well as to provide an abstraction to separate the replicated service from the consumers of the service. The load balancer also provides a resolvable name that is independent of any of the specific replicas.

With Kubernetes, you can create this load balancer with a `Service` object:

```
kind: Service
apiVersion: v1
metadata:
  name: dictionary-server-service
spec:
  selector:
    app: dictionary-server
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
```

Once you have the configuration file, you can create the dictionary service with:

```
kubectl create -f dictionary-service.yaml
```

Session Tracked Services

The previous examples of the stateless replicated pattern routed requests from all users to all replicas of a service. While this ensures an even distribution of load and fault tolerance, it is not always the preferred solution. Often there are reasons for wanting to ensure that a particular user's requests always end up on the same machine. Sometimes this is because you are caching that user's data in memory, so landing on the same machine ensures a higher cache hit rate. Sometimes it is because the interaction is long-running in nature, so some amount of state is maintained between requests. Regardless of the reason, an adaption of the stateless replicated service pattern is to use session tracked services, which ensure that all requests for a single user map to the same replica, as illustrated in Figure 5-3.

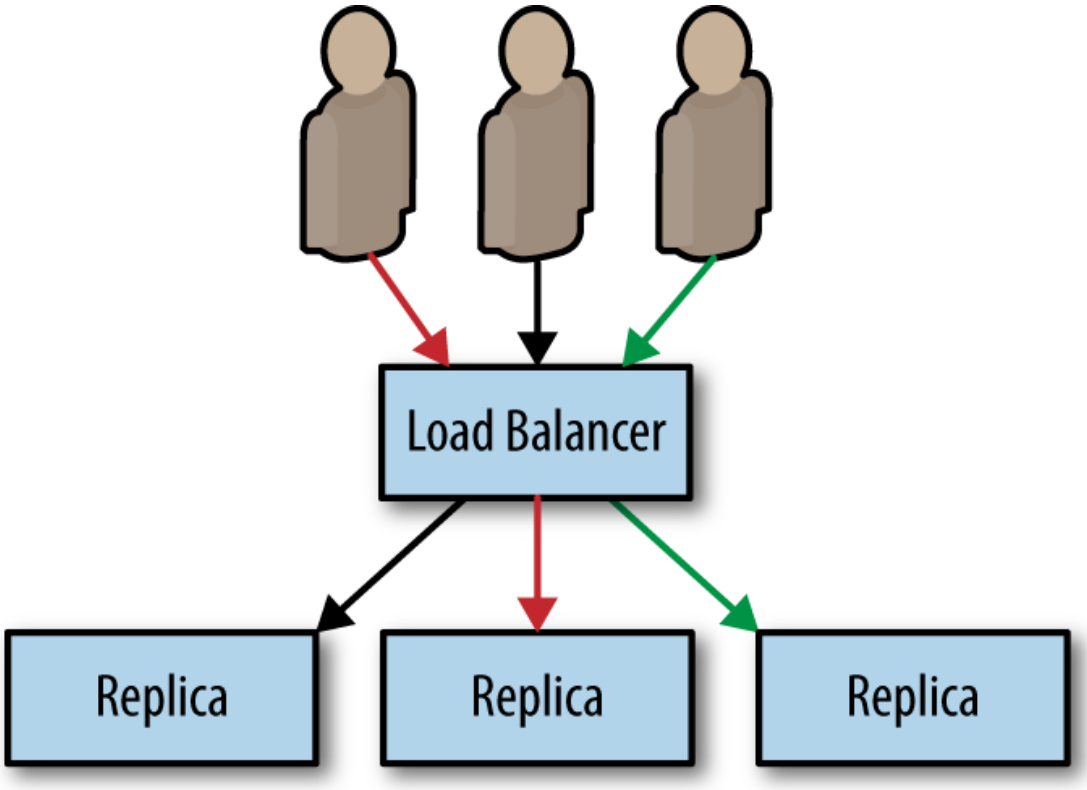

Figure 5-3. A session tracked service where all requests for a specific user are routed to a single instance

Generally speaking, this session tracking is performed by hashing the source and destination IP addresses and using that key to identify the server that should service the requests. So long as the source and destination IP addresses remain constant, all requests are sent to the same replica.

NOTE

IP-based session tracking works within a cluster (internal IPs) but generally doesn't work well with external IP addresses because of network address translation (NAT). For external session tracking, application-level tracking (e.g., via cookies) is preferred.

Often, session tracking is accomplished via a *consistent hashing function*. The benefit of a consistent hashing function becomes evident when the service is scaled up or down. Obviously, when the number of replicas changes, the mapping of a particular user to a replica may change. Consistent hashing functions minimize the number of users that actually change which replica they are mapped to, reducing the impact of scaling on your application.

Application-Layer Replicated Services

In all of the preceding examples, the replication and load balancing takes place in the network layer of the service. The load balancing is independent of the actual protocol that is being spoken over the network, beyond TCP/IP. However, many applications use HTTP as the protocol for speaking with each other, and knowledge of the application protocol that is being spoken enables further refinements to the replicated stateless serving pattern for additional functionality.

Introducing a Caching Layer

Sometimes the code in your stateless service is still expensive despite being stateless. It might make queries to a database to service requests or do a significant amount of rendering or data mixing to service the request. In such a world, a caching layer can make a great deal of sense. A cache

exists between your stateless application and the end-user request. The simplest form of caching for web applications is a caching web proxy. The caching proxy is simply an HTTP server that maintains user requests in memory state. If two users request the same web page, only one request will go to your backend; the other will be serviced out of memory in the cache. This is illustrated in Figure 5-4.

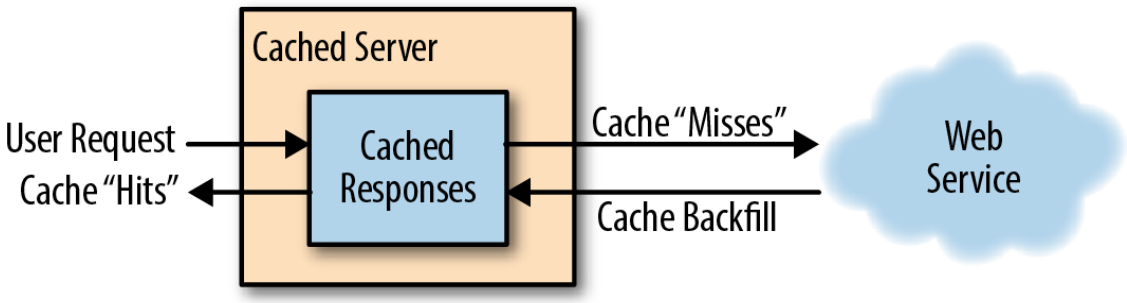

Figure 5-4. The operation of a cache server

For our purposes, we will use Varnish, an open source web cache.

Deploying Your Cache

The simplest way to deploy the web cache is alongside each instance of your web server using the sidecar pattern (see Figure 5-5).

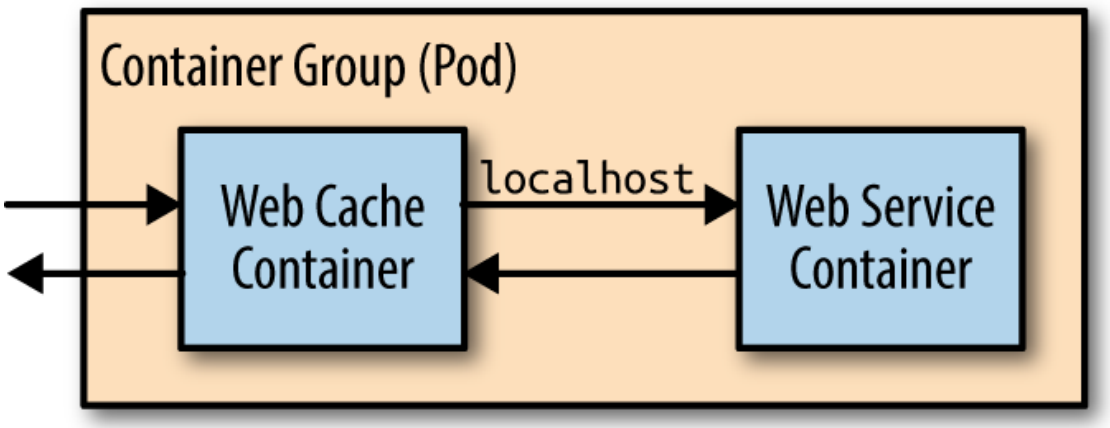

Figure 5-5. Adding the web cache server as a sidecar

Though this approach is simple, it has some disadvantages, namely that you will have to scale your cache at the same scale as your web servers. This is often not the approach you want. For your cache, you want as few replicas as possible with lots of resources for each replica (e.g., rather than 10 replicas with 1 GB of RAM each, you'd want two replicas with 5 GB of RAM each). To understand why this is preferable, consider that every page will be stored in every replica. With 10 replicas, you will store

every page 10 times, reducing the overall set of pages that you can keep in memory in the cache. This causes a reduction in the *hit rate*, the fraction of the time that a request can be served out of cache, which in turn decreases the utility of the cache. Though you do want a few large caches, you might also want lots of small replicas of your web servers. Many languages (e.g., NodeJS) can really only utilize a single core, and thus you want many replicas to be able to take advantages of multiple cores, even on the same machine. Therefore, it makes the most sense to configure your caching layer as a second stateless replicated serving tier above your web-serving tier, as illustrated in Figure 5-6.

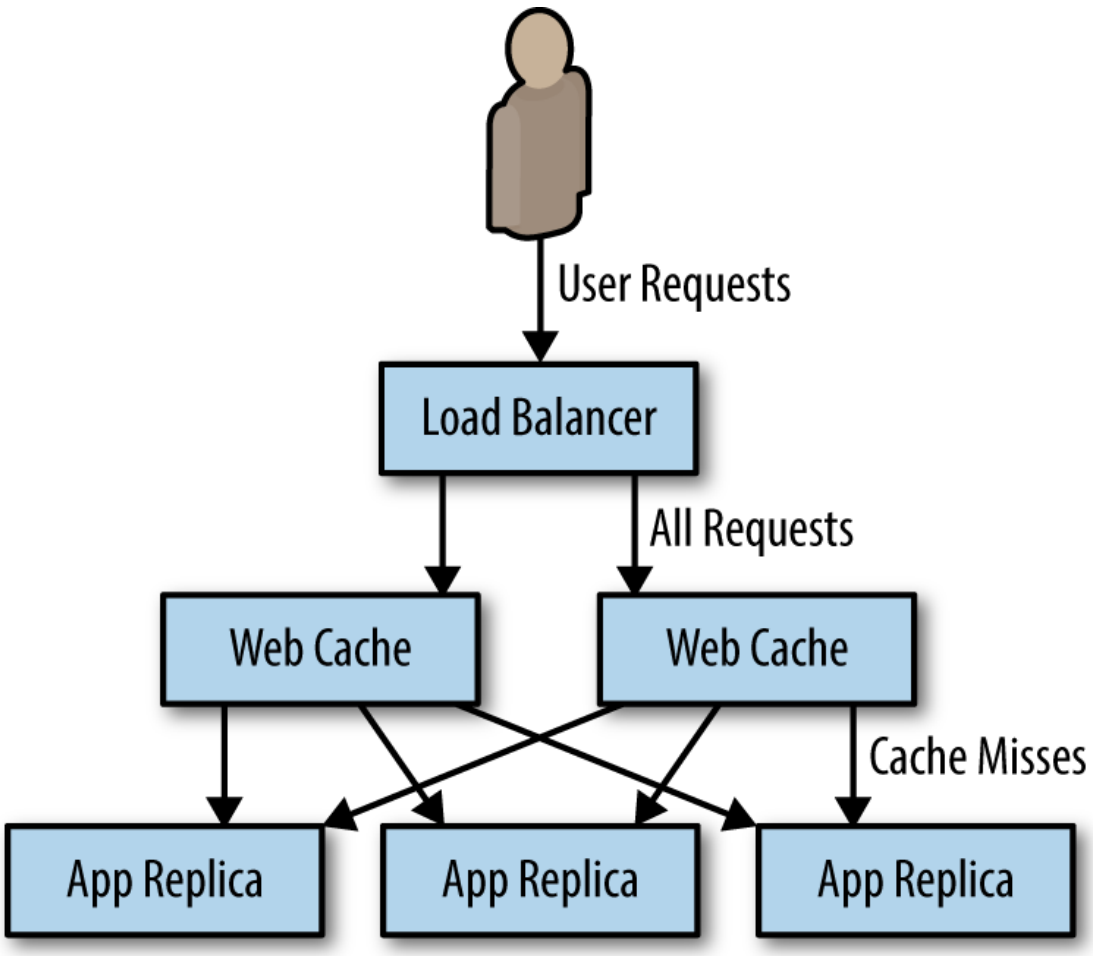

Figure 5-6. Adding the caching layer to our replicated service

---

NOTE

Unless you are careful, caching can break session tracking. The reason for this is that if you use default IP address affinity and load balancing, all requests will be sent from the IP addresses of the cache, not the end user of your service. If you've followed the advice previously given and deployed a few large caches, your IP-address-based affinity may in fact mean that some replicas of your web layer see *no* traffic. Instead, you need to use something like a cookie or HTTP header for session tracking.

Hands On: Deploying the Caching Layer

The dictionary-server service we built earlier distributes traffic to the dictionary server and is discoverable as the DNS name `dictionary-server-service`. This pattern is illustrated in Figure 5-7.

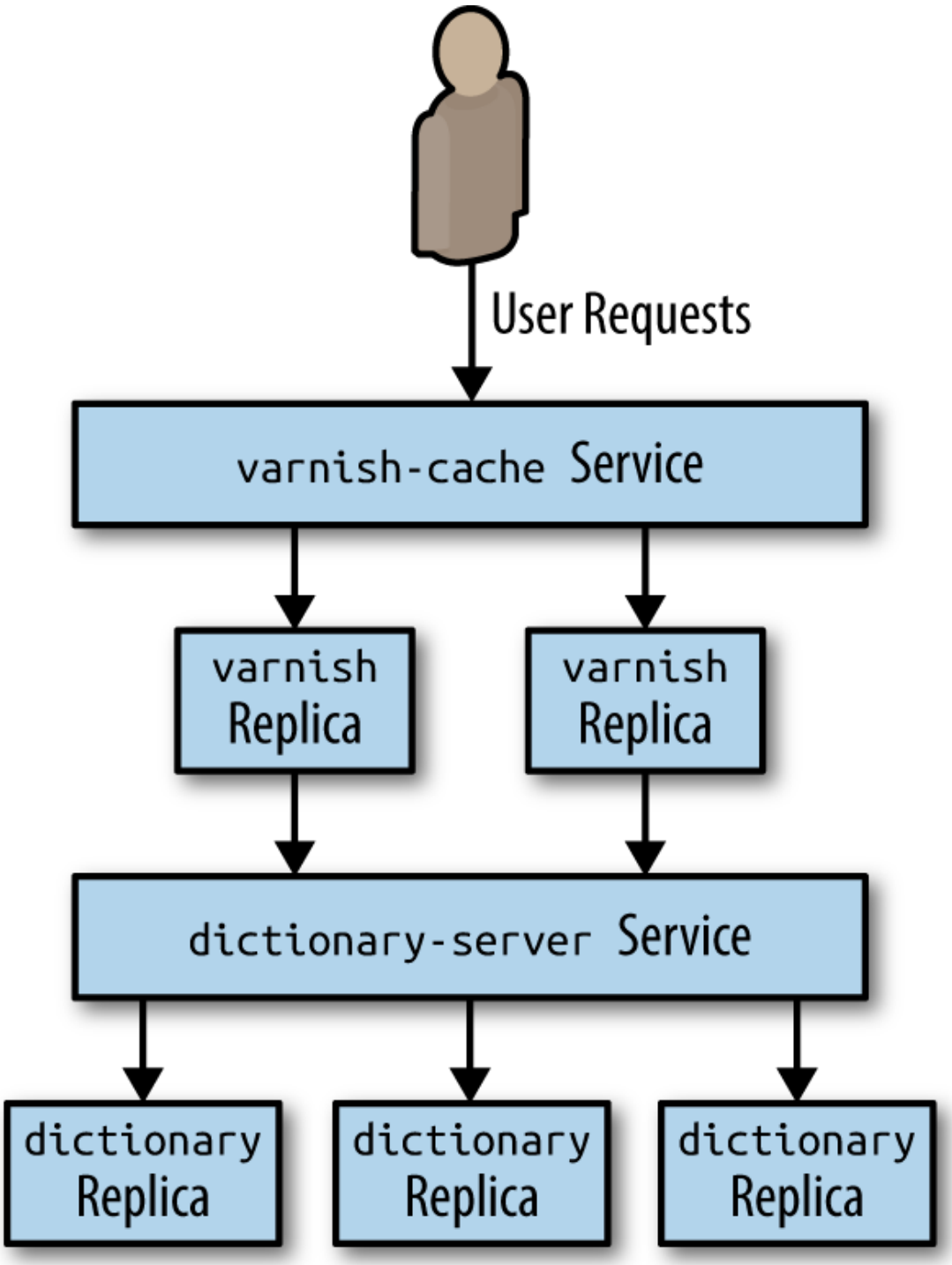

Figure 5-7. Adding a caching layer to the dictionary server

We can begin building this with the following Varnish cache configuration:

```
vcl 4.0;
backend default {
```

```
    .host = "dictionary-server-service";
    .port = "8080";
  }
```

Create a `ConfigMap` object to hold this configuration:

```
kubectl create configmap varnish-config --from-file=default.vcl
```

Now we can deploy the replicated Varnish cache, which will load this configuration:

```
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: varnish-cache
spec:
  replicas: 2
  template:
    metadata:
      labels:
        app: varnish-cache
    spec:
      containers:
      - name: cache
        resources:
          requests:
            # We'll use two gigabytes for each varnish cache
            memory: 2Gi
        image: brendanburns/varnish
        command:
        - varnishd
        - -F
        - -f
        - /etc/varnish-config/default.vcl
        - -a
        - 0.0.0.0:8080
        - -s
        # This memory allocation should match the memory request above
        - malloc,2G
        ports:
        - containerPort: 8080
        volumeMounts:
        - name: varnish
          mountPath: /etc/varnish-config
```

```
      volumes:
      - name: varnish
        configMap:
          name: varnish-config
```

You can deploy the replicated Varnish servers with:

```
kubectl create -f varnish-deploy.yaml
```

And then finally deploy a load balancer for this Varnish cache:

```
kind: Service
apiVersion: v1
metadata:
  name: varnish-service
spec:
  selector:
    app: varnish-cache
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
```

which you can create with:

```
kubectl create -f varnish-service.yaml
```

Expanding the Caching Layer

Now that we have inserted a caching layer into our stateless, replicated service, let's look at what this layer can provide beyond standard caching. HTTP reverse proxies like Varnish are generally pluggable and can provide a number of advanced features that are useful beyond caching.

Rate Limiting and Denial-of-Service Defense

Few of us build sites with the expectation that we will encounter a denial-of-service attack. But as more and more of us build APIs, a denial of service can come simply from a developer misconfiguring a client or a site-

reliability engineer accidentally running a load test against a production installation. Thus, it makes sense to add general denial-of-service defense via rate limiting to the caching layer. Most HTTP reverse proxies like Varnish have capabilities along this line. In particular, Varnish has a `throttle` module that can be configured to provide throttling based on IP address and request path, as well as whether or not a user is logged in.

If you are deploying an API, it is generally a best practice to have a relatively small rate limit for anonymous access and then force users to log in to obtain a higher rate limit. Requiring a login provides auditing to determine who is responsible for the unexpected load, and also offers a barrier to would-be attackers who need to obtain multiple identities to launch a successful attack.

When a user hits the rate limit, the server will return the `429` error code indicating that too many requests have been issued. However, many users want to understand how many requests they have left before hitting that limit. To that end, you will likely also want to populate an HTTP header with the remaining-calls information. Though there isn't a standard header for returning this data, many APIs return some variation of `X-RateLimit-Remaining`.

SSL Termination

In addition to performing caching for performance, one of the other common tasks performed by the edge layer is SSL termination. Even if you plan on using SSL for communication between layers in your cluster, you should still use different certificates for the edge and your internal services. Indeed, each individual internal service should use its own certificate to ensure that each layer can be rolled out independently. Unfortunately, the Varnish web cache can't be used for SSL termination, but fortunately, the `nginx` application can. Thus we want to add a third layer to our stateless application pattern, which will be a replicated layer of `nginx` servers that will handle SSL termination for HTTPS traffic and forward traffic on to our Varnish cache. HTTP traffic continues to travel to the Varnish web cache, and Varnish forwards traffic on to our web application, as shown in Figure 5-8.

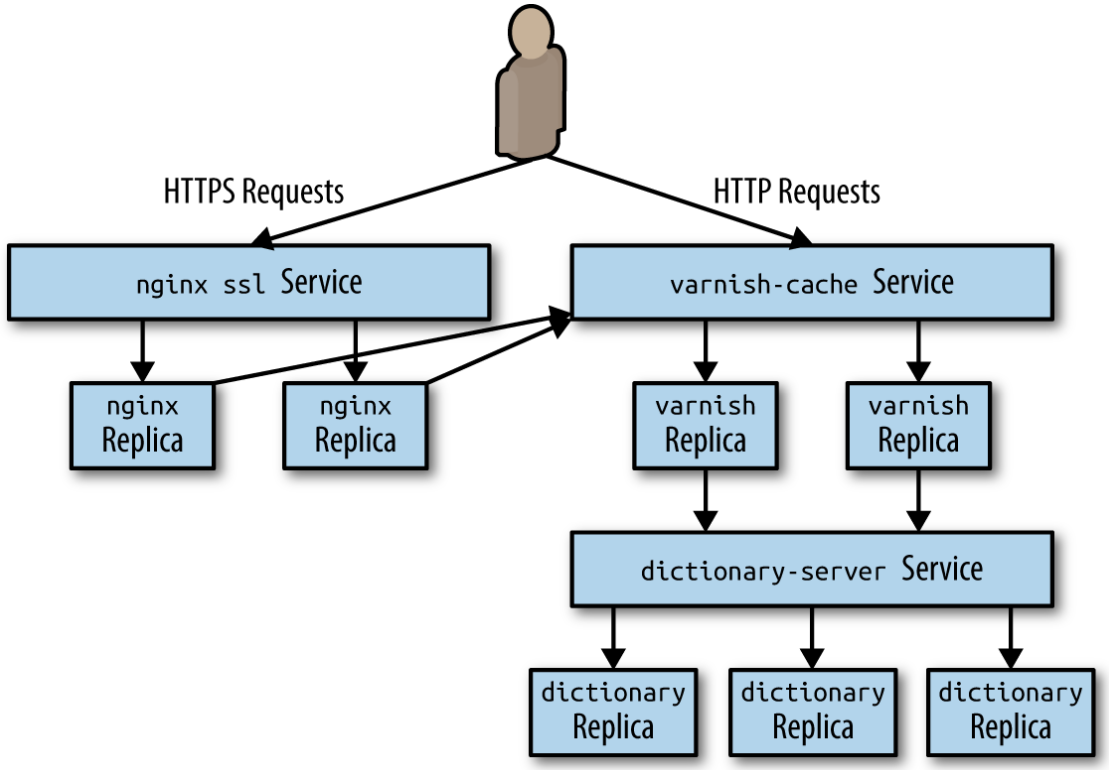

Figure 5-8. Complete replicated stateless serving example

Hands On: Deploying nginx and SSL Termination

The following instructions describe how to add a replicated SSL terminating nginx to the replicated service and cache that we previously deployed.

NOTE

These instructions assume that you have a certificate. If you need to obtain a certificate, the easiest way to do that is via the tools at Let's Encrypt. Alternately, you can use the `openssl` tool to create them. The following instructions assume that you've named them `server.crt` (public certificate) and `server.key` (private key on the server). Such self-signed certificates will cause security alerts in modern web browsers and should never be used for production.

The first step is to upload your certificate as a secret to Kubernetes:

```
kubectl create secret tls ssl --cert=server.crt --key=server.key
```

Once you have uploaded your certificate as a secret you need to create an nginx configuration to serve SSL:

```
events {
  worker_connections  1024;
```

```
}

http {
  server {
    listen 443 ssl;
    server_name my-domain.com www.my-domain.com;
    ssl on;
    ssl_certificate         /etc/certs/tls.crt;
    ssl_certificate_key     /etc/certs/tls.key;
    location / {
        proxy_pass http://varnish-service:80;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Real-IP $remote_addr;
    }
  }
}
```

As with Varnish, you need to transform this into a `ConfigMap` object:

```
kubectl create configmap nginx-conf --from-file=nginx.conf
```

Now that you have a secret and an nginx configuration, it is time to create the replicated, stateless nginx layer:

```
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: nginx-ssl
spec:
  replicas: 4
  template:
    metadata:
      labels:
        app: nginx-ssl
    spec:
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 443
        volumeMounts:
        - name: conf
```

```
            mountPath: /etc/nginx
          - name: certs
            mountPath: /etc/certs
      volumes:
      - name: conf
        configMap:
          # This is the ConfigMap for nginx we created previously
          name: nginx-conf
      - name: certs
        secret:
          # This is the secret we created above
          secretName: ssl
```

To create the replicated nginx servers, you use:

```
kubectl create -f nginx-deploy.yaml
```

Finally, you can expose this nginx SSL server with a service:

```
kind: Service
apiVersion: v1
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx-ssl
  type: LoadBalancer
  ports:
    - protocol: TCP
      port: 443
      targetPort: 443
```

To create this load-balancing service run:

```
kubectl create -f nginx-service.yaml
```

If you create this service on a Kubernetes cluster that supports external load balancers, this will create an externalized, public service that services traffic on a public IP address.

To get this IP address, you can run:

```
kubectl get services
```

You should then be able to access the service with your web browser.

Summary

This chapter began with a simple pattern for replicated stateless services. Then we saw how this pattern grows with two additional replicated load-balanced layers to provide caching for performance, and SSL termination for secure web serving. This complete pattern for stateless replicated serving is shown in Figure 5-8.

This complete pattern can be deployed into Kubernetes using three `Deployments` and `Service` load balancers to connect the layers shown in Figure 5-8. The complete source for these examples can be found at *https://github.com/brendandburns/designing-distributed-systems*.